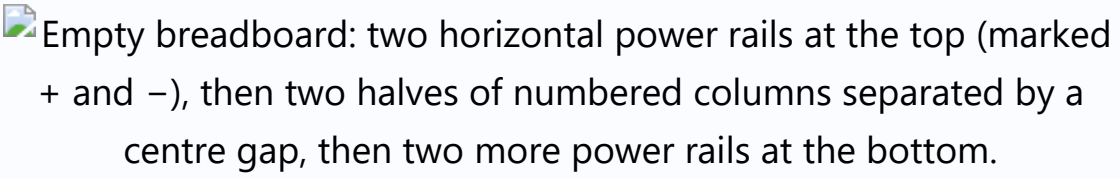
R0 — How a breadboard works

Project 2 • Reaction-Time Game

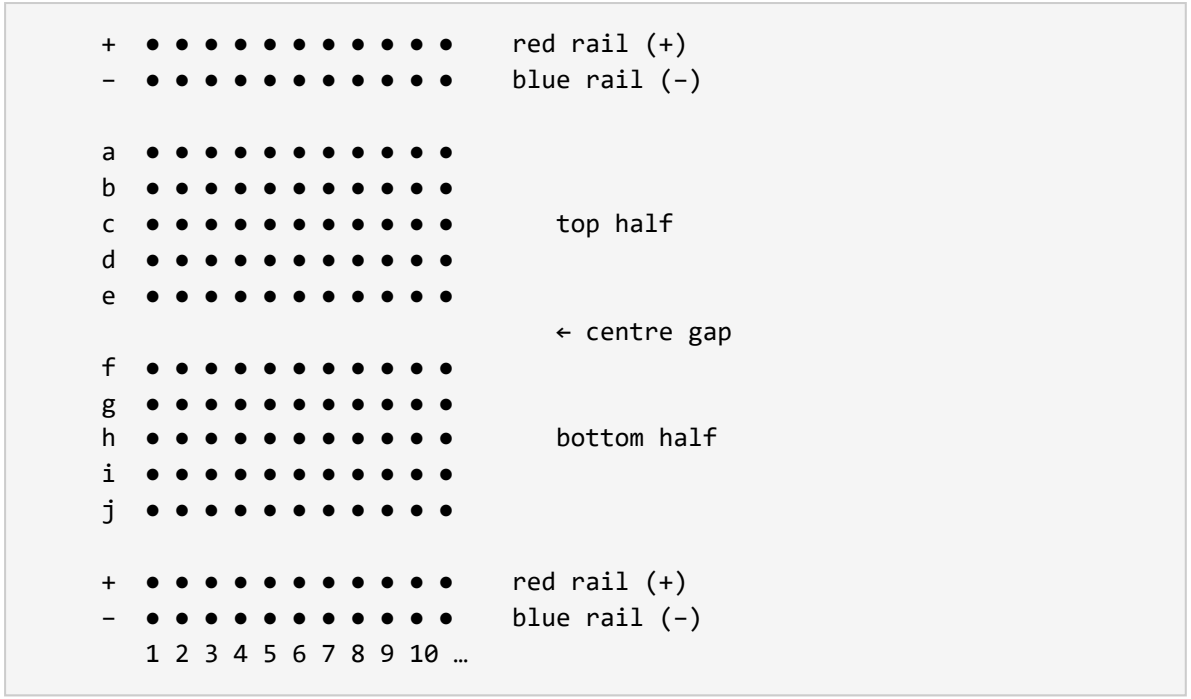
> Read this before your first session. You do not need to memorise it — keep it on your workbench and come back to any section when you get stuck.

A breadboard lets you build circuits without soldering. You push the legs of parts (LEDs, resistors, the buzzer, wires) into the holes, and the board connects them together inside — no tools needed. When you are done, you can pull everything out and reuse the parts.

The board has four areas



The real breadboard. Two red/blue **power rails** at top and bottom (marked + and −). In between, a grid of numbered columns split by a **centre gap**. Rows are labelled **a–e** on the top half and **f–j** on the bottom half.



The two rules that matter

- **Inside one numbered column, rows a–e are connected.** Holes a5, b5, c5, d5, e5 all share the same electrical connection. Anything you push into those 5 holes is wired together.
- **Rows f–j are connected the same way, but separately from a–e.** The centre gap breaks the connection — holes e5 and f5 are **not** connected to each other.

The two power rails

- The **red** rail (marked +) runs the full length of the board. Every hole along the red rail is connected. Used for **5 V**.
- The **blue** rail (marked −) also runs the full length. Every hole along the blue rail is connected. Used for **GND** (ground).
- Because each rail runs the full length, you can tap power from anywhere along it — you do not need a separate wire for each LED.

How to place an LED on the breadboard

An LED has **two legs of different lengths**. The long leg is the plus side; the short leg is the minus side. Each leg needs its **own numbered column** — if both legs go into the same column, the LED is short-circuited and will not light.

> **Rule of thumb:** plant the LED so its two legs go into two different numbered columns (for example, long leg in column 9, short leg in column 8). Resistors work the same way — legs in two different columns.

What "pin 9 → 220 Ω → LED" means on the breadboard

When reference card **R1 (Project 2 wiring)** writes "*Arduino pin 9 → 220 Ω → LED long leg*", here is the chain of connections you are building:

```
[Arduino pin 9] ── wire ── [resistor] ── [LED long leg] ──
                [LED short leg] ── wire ── [GND]
```

On the breadboard, each arrow above is one shared column. Using the exact layout from R1 (Project 2 wiring):

- A jumper wire from Arduino pin 9 plugs into one hole on the breadboard.
- One leg of the 220 Ω resistor plugs into the **same numbered column** as that jumper — they are now connected.
- The resistor's other leg plugs into a **new column** — a fresh, separate connection.
- The LED's long leg plugs into that same new column — now current can flow from pin 9 → resistor → LED.
- The LED's short leg plugs into its own column, and a jumper from that column goes to the blue GND rail.

The buzzer — the new part in Project 2

- The buzzer connects in a short chain: **pin 8** → **buzzer** → **GND**. One buzzer leg to digital pin 8, the other leg to the GND rail.
- The small (passive piezo) buzzer **needs no resistor**, and it does not matter which way round you connect its legs.
- Like every part — the two legs go into **two different columns**, or it shorts and the buzzer will not beep.
- **Never wire the buzzer directly between 5 V and GND.** Always drive it from pin 8, so the Arduino controls it.

The breadboard is doing the wiring for you — you just pick which column each leg goes into.

Two things to check if something is not working yet

Check 1 — are the LED's two legs in two different columns? If both legs share a column, the long leg and the short leg are connected directly, which shorts the LED and it will not light. Pull one leg and move it one column over. The same is true for the buzzer — its two legs in different columns.

Check 2 — does each leg share its column with exactly the right neighbour? Trace the chain out loud: "pin 9 jumper and resistor leg go into the *same* column; the resistor's other leg and the LED long leg go into the *same* column; the LED short leg and the GND jumper go into the *same* column." If any of those pairs are in *different* columns, the chain is broken there. Move the leg into the matching column.

You have what you need

Your next card is **R1 — Project 2 wiring**. Keep R0 on your workbench — if something is not working later, the two checks above will usually find it.

R1 — Wiring Reference

Project 2 • Reaction-Time Game • Student workstation card

This card shows you how to wire the LED, the button, and the buzzer for Project 2. Keep it next to your breadboard. If you get confused about wiring, look at this card first before you call the teacher.

⚠️ The most important rules

LED long leg = positive (+). It goes to the resistor, then to the Arduino pin.

LED short leg = negative (−). It goes to the Arduino GND.

Every LED needs its own 220 Ω resistor (colour bands: `red · red · brown`). No resistor = burnt LED.

The button needs a 10 kΩ pull-down resistor (colour bands: `brown · black · orange`). Without it the button reads random values.

The buzzer has no polarity and needs no resistor. One leg goes to digital pin 8, the other to GND. Never connect the buzzer straight to 5 V.

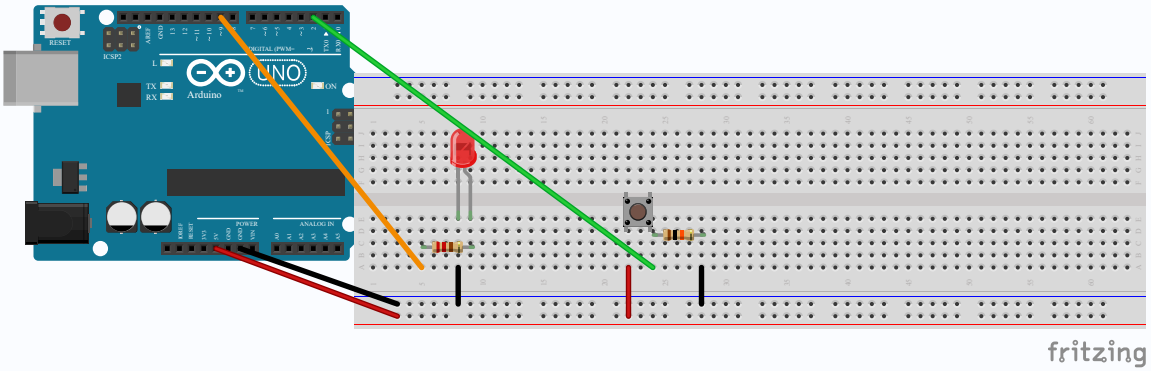
🔌 Wiring 1 — LED + button (Milestone 1)

The base circuit of the game: one LED on **pin 9** through a 220 Ω resistor, and a button on **pin 2** with a 10 kΩ pull-down resistor. This is exactly the circuit you built in Project 1.

```
Arduino pin 9 ──[220 Ω]──┬── LED long leg (+)
                         ▼
```

```
                        LED short leg (–) ── Arduino GND

Arduino 5 V ──────────────────┬── Button pin A
                              ▼
                        Button pin B ──┬── Arduino pin 2
                                       └─[10 kΩ]── Arduino GND
```



LED: pin 9 → 220 Ω → long leg, short leg → **GND**. **Button: pin A → 5 V**, pin B → pin 2 and through a **10 kΩ pull-down → GND**.

Important: the 10 kΩ pull-down resistor goes between **pin 2 and GND**, **not between 5 V and GND**. If you wire it between 5 V and GND, pressing the button creates a short circuit and the Arduino will reset.

🔌 Wiring 2 — LED + button + buzzer (Milestone 4)

Same as Wiring 1, plus the **buzzer**: one leg to **pin 8**, the other leg to **GND**. The buzzer's two legs go in **different breadboard columns** — not the same column, which would short it.

```
Arduino pin 9 ──[220 Ω]──┬── LED long leg (+)
                         ▼
                        LED short leg (–) ── Arduino GND

Arduino 5 V ──────────────────┬── Button pin A
                              ▼
                        Button pin B ──┬── Arduino pin 2
                                       └─[10 kΩ]── Arduino GND

Arduino pin 8 ──────────────── Buzzer leg 1
Arduino GND  ──────────────── Buzzer leg 2
```

Project 2 · Circuit 2 — LED + Button + Buzzer

Adds the piezo buzzer · pin map: LED=D9, Button=D2, Buzzer=D8

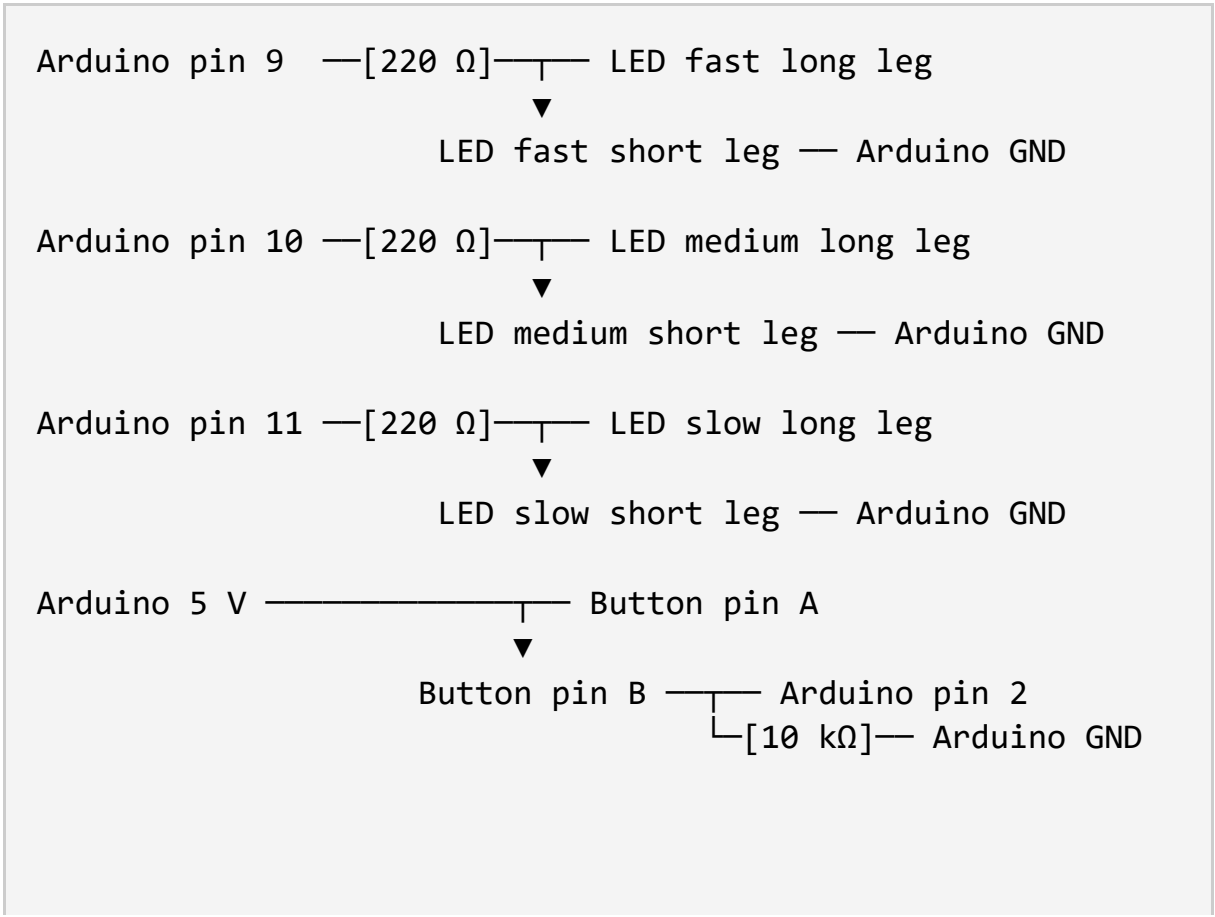
- **Arduino D9** —[220 Ω]— LED long leg (+) ; short leg (−) — **GND**
- **Arduino 5 V** — Button pin A
- Button pin B — **Arduino D2** · pin B —[10 kΩ]— **GND**
- **Arduino D8** — Buzzer leg 1
- Buzzer leg 2 — **GND** (small passive piezo — no resistor; drive only from D8)

Schematic (preliminary). A photo-style breadboard image will replace this diagram.

Buzzer: one leg → pin 8, the other leg → **GND**. The buzzer is added on top of Wiring 1 — no polarity, no resistor. **Never connect the buzzer to 5 V.**

🔌 Wiring 3 — Three LEDs + button + buzzer (Tier 2, feedback Mode A)

Only needed if you chose the **three-LED feedback mode** in Tier 2 (Milestone 2b). Add two more LEDs to Wiring 2: the **medium LED on pin 10** and the **slow LED on pin 11**, each with its own 220 Ω resistor, exactly like the "go" LED on pin 9.



```
Arduino pin 8 ──────────────── Buzzer leg 1
Arduino GND  ──────────────── Buzzer leg 2
```

Project 2 · Circuit 3 — Three LEDs + Button + Buzzer

Tier 2 three-LED feedback · LEDs=D9/D10/D11, Button=D2, Buzzer=D8

- **D9** —[220 Ω]— FAST LED (green) — short leg — **GND**
- **D10** —[220 Ω]— MEDIUM LED (yellow) — short leg — **GND**
- **D11** —[220 Ω]— SLOW LED (red) — short leg — **GND**

- **5 V** — Button pin A · pin B — **D2** · pin B —[10 kΩ]— **GND**

- **D8** — Buzzer leg 1 · leg 2 — **GND**

Schematic (preliminary). A photo-style breadboard image will replace this diagram.

Pin 9 → **fast LED** · **Pin 10** → **medium LED** · **Pin 11** → **slow LED**, each through a 220 Ω resistor to GND. Added to the button (pin 2) and buzzer (pin 8) of Wiring 2.

Wiring 4 — Two-button variant (Tier 3 only)

Only needed if you are building the **two-player game** in Tier 3. Add a **second button** on **pin 3** with its own 10 kΩ pull-down resistor, wired exactly like the first button.

```
Arduino 5 V ──────────────┬── Button 1 pin A
                          ▼
                  Button 1 pin B ──┬── Arduino pin 2
                                   └─[10 kΩ]── Arduino GND

Arduino 5 V ──────────────┬── Button 2 pin A
                          ▼
                  Button 2 pin B ──┬── Arduino pin 3
                                   └─[10 kΩ]── Arduino GND
```

Same pull-down rule: each button's 10 kΩ resistor goes between **its input pin (2 or 3) and GND**, not between 5 V and GND.

Project 2 · Circuit 4 — Two-Button Variant (Tier 3)

Head-to-head game · Button 1=D2, Button 2=D3, LED=D9, Buzzer=D8

- **D9** —[220 Ω]— LED (+) ; short leg — **GND** · **D8** — Buzzer — **GND**
- **Player 1:** 5 V — Button 1 pin A ; pin B — **D2** ; pin B —[10 kΩ]— **GND**
- **Player 2:** 5 V — Button 2 pin A ; pin B — **D3** ; pin B —[10 kΩ]— **GND**

Each player has their own button; the sketch shows who pressed first.

Schematic (preliminary). A photo-style breadboard image will replace this diagram.

Button 1 → pin 2 · **Button 2 → pin 3**, each with a **10 kΩ pull-down → GND**. Same pattern as the first button, different pin. Added to the LED and buzzer of the base circuit.

R2 — Stuck? Try This First

Project 2 • Student workstation card

If you get stuck on any task card, these four steps are worth trying — steps 1 or 2 solve most cases. If you would rather call the teacher instead, step 4 is always open.

1

Re-read the step

Read the task card's current step again, slowly. Sometimes the second reading answers the question.

2

Check the wiring reference card (R1)

About 60% of "my LED isn't working" moments are wiring mistakes. R1 shows you exactly how the Project 2 circuit should look.

3

Check the other reference cards

R3 (Claude Code prompts), R4 (safety), R5 (which sketch file to open). One of them may answer your question.

4

Call the teacher

Raise your hand or say the teacher's name. The teacher will come to you. (You do not need a cup or a flag — just call.)

You are not interrupting. Calling the teacher when you are stuck is exactly what the teacher expects you to do. Being stuck is not a failure

— it is a normal part of learning Arduino, and figuring out *what* you are stuck on is useful information for the next step.

R3 — Claude Code Prompt Templates

Project 2 • How to talk to Claude Code

Claude Code is your coding collaborator. It runs in a window pointed at your Project 2 folder, so it can see the sketches you are working on. There are two ways to use it.

Channel A — When you want help with code (Level 2)

Before you ask Claude Code anything about your code, **fill in these three things on your task card**:

> **(a)** What I want to happen: *[describe the behaviour you want]*
> **(b)** What is currently happening: *[describe what the sketch does now]*
> **(c)** What I have tried: *[what you have already looked at or considered]*

Then use this prompt template:

```
I am working on [sketch file name].

(a) What I want to happen:
[your answer]

(b) What is currently happening:
[your answer]

(c) What I have tried:
[your answer]

Can you help me figure out what to change?
```

The (a)(b)(c) is not optional. It helps you think about the problem before Claude Code answers, and it helps Claude Code give you a useful answer. Skipping it usually gets a worse answer.

After Claude Code answers, you make the change in the Arduino IDE, upload it, and test. Then answer this question on your task card:

> **Comprehension check:** In one sentence, what did Claude Code change?

A worked Level 2 example from Project 2 — change the LED time

In Project 2 the real code change is setting how long the LED stays on: **2 seconds** (easy) or **0.5 seconds** (hard). Here is how you fill in the three things before you prompt:

> **(a)** What I want to happen: *The LED should stay on for 0.5 seconds instead of 2 seconds.*
> **(b)** What is currently happening: *The LED stays on for 2 seconds.*
> **(c)** What I have tried: *I looked for the line with the delay number but I am not sure which line it is.*

Then use this short template, and paste your current code where it says [paste]:

```
Here is my current delay. How do I make the LED stay on for
only 0.5 seconds instead of 2 seconds? Here is my code:
[paste]
```

It is one line in the code. After Claude Code answers, change only that number in the Arduino IDE, upload, and check that the game feels different.

Channel B — When you want the task card read to you

Channel B walks you through your current task card conversationally. Use it when the printed card feels hard to read, or when you prefer hearing instructions one step at a time. Use this prompt:

```
I'm on Project 2, Tier [1 or 2], Milestone [number].
Walk me through it.
```

Claude Code will read the card's content in small pieces and ask you to confirm each step before moving on. You still check off the printed card as you go — Channel B does not replace the card, it just helps you work through it.

> **Channel B is not recommended for Milestone 1 of any project, because Milestone 1 is a together-milestone where the teacher is next to you. Hearing Claude Code speak while the teacher is also speaking is confusing.**

Channel A Level 3 — Free dialogue (Tier 3 only)

If you are working at Tier 3 on an open-design project, you can talk to Claude Code freely about what you are trying to build. The same (a)(b)(c) discipline still applies — describe what you want before you ask.

R4 — Safety Reminder

Project 2 • Reaction-Time Game

Project 2 is very safe too. The Arduino runs on 5 volts, which is too low to hurt you, and the new buzzer you add in this project is completely safe as well. But you can still break components if you wire something wrong. Read these three rules and you will be fine.

Rule 1 — Never connect 5 V directly to GND

> **Do not** put a jumper wire straight from **5 V** to **GND**. This is called a *short circuit*. It will make the Arduino reset itself or turn off to protect itself. No one gets hurt, but the circuit stops working until you remove the bad wire.

The 5 V pin powers the LEDs, sensors, and buttons you connect. The GND pin is the return path for the electricity. They should never touch each other directly.

Rule 2 — Every LED needs a resistor, and the buzzer has one right hook-up

> **Do not** connect an LED directly to a 5 V pin with no resistor. The LED will light up very brightly for a second or two, get hot, and burn out. Every LED in Project 2 has a **220 Ω resistor** (colour bands: red, red, brown) between its long leg and the Arduino pin.

> If you see an LED in your circuit without a resistor, fix it before you upload anything.

> The buzzer in Project 2 is a **small passive piezo buzzer**, and it is safe when you drive it from **digital pin 8** with the `tone()` command — one leg to pin 8, the other leg to **GND**. No resistor is needed. **Do not** connect the buzzer directly between **5 V** and **GND** — that would make it buzz non-stop and can damage it.

Rule 3 — USB cable has one right way

> The USB cable's square end fits into the Arduino only one way. If it does not go in, do not push harder — you will break the connector. Flip the plug over and try again.

If something stops working

If something stops working, the most useful next move is to pause and look at the wiring — more button-pressing usually does not help. If you cannot see what is wrong, call the teacher. That is what the stuck protocol (R2) is for. Wiring things wrong is part of learning Arduino.

R5

💻 Which file to open at which milestone

Project 2 • Sketch Index

Where your Project 2 files live

All your Project 2 sketches are in your Project 2 folder on Google Drive. To get there: in the File Explorer sidebar click **Google Drive**, go to **My Drive** → **Arduino_Projects**, open your nickname folder, then open `Project_2_Reaction_Time_Game` and inside it `ino_files`.

If you cannot find the folder, ask the teacher once and write down how to get there here: _____

Inside `ino_files` each sketch sits in its own folder. Each folder contains one `.ino` file with the same name as the folder.

> Open a sketch by double-clicking the `.ino` file, or by using **File** → **Open** in the Arduino IDE and pointing at the file.

Tier 1 sketches — which one for which milestone

Milestone	Open this file	What happens
M2	`01_wait_flash_measure/01_wait_flash_measure.ino`	The Arduino waits a few random seconds, turns on the LED (pin 9), measures how fast you press the button, and prints your reaction time on the Serial Monitor
M5	`02_wait_flash_measure_buzzer/02_wait_flash_measure_buzzer.ino`	The same game, but now the buzzer (pin 8) beeps at the "go" moment and plays a success tone on the press — on top of the Serial Monitor reading

M1, M3, M4, M6 — no sketch upload. M1 and M4 are hardware only (wiring the LED and button, and adding the buzzer). M3 and M6 are playing the game and recording your fastest time.

Tier 2 starter sketches — pick one for your feedback mode

The feedback mode you chose	Open this file
Mode A — Three LEDs (fast / medium / slow)	`T2_three_led_feedback_starter/T2_three_led_feedback_starter.ino`

The feedback mode you chose	Open this file
Mode B — Buzzer pattern	`T2_buzzer_pattern_starter/T2_buzzer_pattern_starter.ino`
Mode C — Serial Monitor readout	`T2_serial_readout_starter/T2_serial_readout_starter.ino`

At Tier 2, you upload the starter sketch first to see it work, then use Claude Code (see Ⓡ3) to modify it — for example to shorten how long the LED stays on — so it matches your own design.

Tier 3 — no starter sketch

At Tier 3 you design your own game from scratch with Claude Code (for example a two-player game or a multi-round scored game). Your finished sketch lives in the same folder — pick any name you like (something like `my_reaction_game.ino`).

V1

Project #2 — Reaction-Time Game: Milestone 1 of 6

🔌 Wire the LED and button

> *In this milestone you rebuild the Project 1 base circuit — an LED and a button. This is the foundation of the reaction-time game: the LED turns on, and you press the button as fast as you can.*

📋 What to do

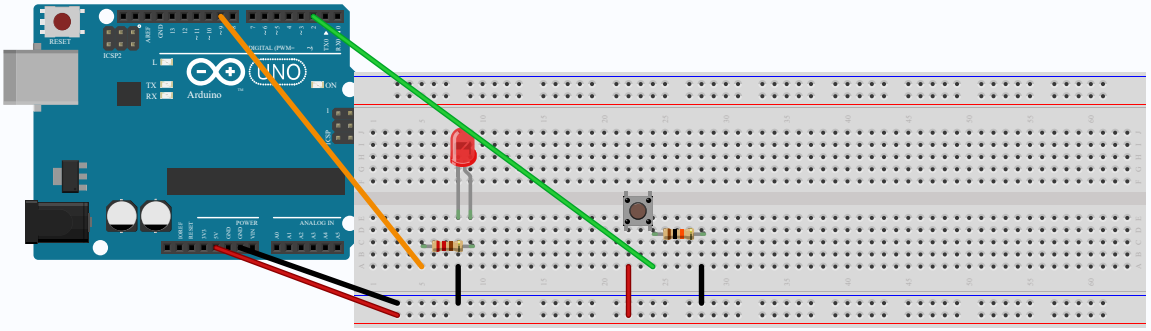
- [ ] In the File Explorer sidebar click **Google Drive**, then go to **My Drive** → **Arduino_Projects**.
- [ ] Open your nickname folder.
- [ ] Create a new folder inside it called **Project_2_Reaction_Time_Game**.
- [ ] Open Claude Code together with the teacher and point it at your new folder.
- [ ] Pick one LED from the parts tray. Any colour is fine.
- [ ] **The LED:** plant it in the breadboard so its two legs land in different rows (not the same row — that would short them). The **long leg is positive (+)**, the **short leg is negative (−)**.
- [ ] Connect the long leg (+) to **digital pin 9** through a **220 Ω resistor** (bands: **red · red · brown**), and the short leg (−) to **GND** with a jumper wire.
- [ ] **The button:** pick a tactile push-button (four pins) and plant it **across the centre gap** of the breadboard. If it does not sit right, rotate it 90 degrees.
- [ ] Connect one side of the button (pin A) to **5 V**, and the other side (pin B) to **digital pin 2**.

- [ ] From pin B, also connect through a **10 kΩ resistor** (bands: **brown · black · orange**) to **GND**. This is the pull-down resistor — it holds pin 2 at 0 V when the button is not pressed.

Wiring diagram

```
Arduino pin 9 ──[220 Ω]──┬── LED long leg (+)
                         ▼
                   LED short leg (–) ── Arduino GND

Arduino 5 V ──────────────┬── Button pin A
                          ▼
                   Button pin B ──┬── Arduino pin 2
                                  └─[10 kΩ]── Arduino GND
```



LED: pin 9 → **220 Ω** → **long leg**, short leg → GND. **Button: pin A** → **5 V**, pin B → pin 2 and through a **10 kΩ pull-down** → **GND**.

⚠ IMPORTANT — THE MOST COMMON MISTAKE

The 10 kΩ pull-down resistor must go between **PIN 2 AND GND**, not between 5 V and GND. If you put it between 5 V and GND, pressing the button creates a short circuit and the Arduino resets itself. Trace it with your finger: does it go from pin 2 to GND? Then you are safe. Call the teacher if you would like help.

👀 WHAT YOU SHOULD SEE

The LED is wired to pin 9 through its resistor, and the button straddles the centre gap — one side to 5 V, the other to pin 2 and through the pull-down resistor to GND.

NOTHING LIGHTS UP OR HAPPENS YET

— you have not uploaded a sketch. That comes at Milestone 2. Right now you are just checking the wiring is correct.

✅ DONE WHEN

- The LED is on the breadboard with its legs in different rows, long leg to pin 9 through a 220 Ω resistor, short leg to GND
- The button straddles the centre gap: pin A to 5 V, pin B to pin 2
- A 10 kΩ resistor sits between pin 2 and GND (the pull-down)
- The teacher has confirmed the wiring is correct

🪄 STUCK?

You did this wiring in Project 1, but if you are unsure, call the teacher. If the Arduino keeps resetting or turning off, the pull-down resistor is almost certainly on the wrong side.

V1

💻 Upload the wait-flash-measure sketch

> *In this milestone you upload a ready-made sketch that runs the game: the LED turns on after a wait, you press fast — and the code measures how fast.*

📋 What to do

- [ ] Open the file `01_wait_flash_measure.ino` in the Arduino IDE.

  The file is in your Project 2 folder under `ino_files/01_wait_flash_measure/`. If you are not sure which file to open, check reference card **R5**.

- [ ] **Do not change any line of the code.** Just open it and upload.

- [ ] Click the Upload button — the right-arrow button (→). Watch the progress bar at the bottom of the IDE.

- [ ] When upload finishes, you should see the green message **"Done uploading"** at the bottom.

- [ ] Open the **Serial Monitor**: click the **magnifying-glass icon** in the top-right of the IDE. Make sure the baud rate is set to **9600**.

- [ ] Read the messages in the Serial Monitor, wait for the LED to light up, and press the button as fast as you can. After your first press, your reaction time appears **in milliseconds (ms)**.

👀 WHAT YOU SHOULD SEE

The Serial Monitor shows the game's instructions, then the LED lights up, and after your press your reaction time appears in milliseconds. The Arduino IDE shows "Done uploading" in green at the bottom.

The number is how many milliseconds it took you to press after the LED lit — the smaller it is, the faster you reacted.

✅ DONE WHEN

- The Arduino IDE shows "Done uploading" in green
- The Serial Monitor is open and shows a reaction time after your first press

🪄 STUCK?

If nothing seems to happen after upload: the Serial Monitor is probably not open. Click the magnifying-glass icon and check the baud is set to 9600 — that is where the game's instructions appear.

If Upload fails: check that Tools → Board shows "Arduino Uno" and Tools → Port shows a COM port. If the Serial Monitor is open, close it, upload again, then reopen it.

If the IDE says "Arduino not found," unplug the USB cable, wait three seconds, plug it back in.

If still stuck after that, call the teacher.

V1

🎮 Play the game five times

> *This is the moment the game works and it is fun — wait for the LED, press as fast as you can, and watch your reaction time appear on the screen.*

📋 How to play — one round

- [ ] Make sure the **Serial Monitor** is open. That is where the instructions and your reaction time show up.
- [ ] **Wait.** The LED is off. Do not press yet — the wait time is different each round, so you cannot guess when.
- [ ] **The LED turns on — press the button as fast as you can!**
- [ ] Read your **reaction time** on the Serial Monitor. It is in milliseconds (ms) — the smaller the number, the faster you reacted.

🏆 Now play five rounds

- [ ] Repeat the round **five times**.
- [ ] Each round, **try to beat your own best** — get a smaller number than last time.
- [ ] Notice: the wait before the LED turns on is **different every round** — the Arduino picks a random number. That is what makes the game fair.

👀 WHAT YOU SHOULD SEE

Each round: the LED is off, then turns on after a different wait. The moment you press, your reaction time appears on the Serial Monitor in milliseconds.

Your reaction times change from round to round — that is completely normal. The wait before the LED also changes each time, because it is random.

✅ DONE WHEN

- You have played five full rounds
- You can point on the Serial Monitor to your **FASTEST** reaction time

🪄 **STUCK? GAME NOT RESPONDING? TRY THIS FIRST:**

Is the Serial Monitor open? Without it you see nothing. Open it from the magnifying-glass icon in the Arduino IDE.

LED never turns on? Check that the Milestone 2 upload worked ("Done uploading" in green) and that the LED is the right way round — long leg (+) to the resistor and on to pin 9.

Press not counted? Most likely the 10 kΩ pull-down resistor is not between pin 2 and GND. Check the wiring against Milestone 1.

If still stuck, call the teacher.

V1

🔊 Add the buzzer

In this milestone you add a buzzer to the circuit. Soon the game will be able to make a sound as well as a light — so you can react to sound as well as light.

📋 What to do

- [ ] Pick a small piezo buzzer from the parts tray. It is the round, flat one with two legs.
- [ ] Plant the buzzer in the breadboard so its two legs land in different columns — not the same column, that would short it.
- [ ] Connect one leg of the buzzer to **digital pin 8** with a jumper wire.
- [ ] Connect the other leg of the buzzer to **GND** with a jumper wire.
- [ ] This small buzzer needs no resistor, and it has no positive or negative side — either leg can go to either place.

Wiring diagram

```
Arduino pin 8 ──────────┬── Buzzer leg 1
                        ▼
                  Buzzer leg 2 ── Arduino GND
```

Project 2 · Circuit 2 — LED + Button + Buzzer

Adds the piezo buzzer · pin map: LED=D9, Button=D2, Buzzer=D8

- **Arduino D9** —[220 Ω]— LED long leg (+) ; short leg (−) — **GND**
- **Arduino 5 V** — Button pin A
- Button pin B — **Arduino D2** · pin B —[10 kΩ]— **GND**
- **Arduino D8** — Buzzer leg 1
- Buzzer leg 2 — **GND** (small passive piezo — no resistor; drive only from D8)

Schematic (preliminary). A photo-style breadboard image will replace this diagram.

Buzzer: one leg → pin 8, the other leg → **GND**. The LED and button stay exactly as in Milestone 1.

⚠️ **IMPORTANT — DO NOT WIRE IT STRAIGHT TO 5 V**

Never connect the buzzer directly between **5 V AND GND**. Always drive it from **PIN 8**, so the Arduino controls it. A buzzer wired straight to 5 V will sound non-stop and can draw too much current. Trace it with your finger: does one leg go to pin 8? Then you are safe. Call the teacher if you would like help.

👀 WHAT YOU SHOULD SEE

The breadboard now has an LED, a button, and a buzzer. One leg of the buzzer goes to pin 8, the other to GND.

THE BUZZER DOES NOT BEEP YET

— you have not uploaded a sketch that drives it. That comes at Milestone 5. Right now you are just checking the wiring is correct.

✅ DONE WHEN

- The buzzer is on the breadboard with its two legs in different columns
- One leg of the buzzer is connected to pin 8
- The other leg of the buzzer is connected to GND
- The teacher has confirmed the wiring is correct

🪄 STUCK?

If both buzzer legs are in the same column, move one to another column. Check that one leg really reaches pin 8 (not a different pin) and the other is in GND. If you are unsure, call the teacher.

V1

💻 Upload the updated sketch with buzzer feedback

> *In this milestone you upload a new version of the code — the same game exactly, but now the buzzer beeps at the "go" moment and plays a short success tone when you press the button.*

📋 What to do

- [ ] Open the file `02_wait_flash_measure_buzzer.ino` in the Arduino IDE.
  It is in your Project 2 folder, next to the sketch you uploaded before.
- [ ] Do not change anything in the code — just upload it.
- [ ] Click the Upload button.
- [ ] Wait for "Done uploading" in green.
- [ ] Play one round: wait for the "go", hear the beep, and press the button as fast as you can.

> 👀 **WHAT YOU SHOULD SEE**
>
> Every round, the buzzer beeps the moment the LED turns on (the "go"), then plays a short tone the moment you press the button. The reaction time is still printed on the Serial Monitor, just like before — now the game can be heard, not only seen.

✅ DONE WHEN

- "Done uploading" showed in green in the IDE
- In a round, the buzzer beeps at the "go" moment
- A short tone plays the moment you press the button

🪄 STUCK?

No beep at all: the old sketch may still be on the board — make sure you uploaded `02_wait_flash_measure_buzzer.ino` and not the previous file, then upload again.

Still no beep after re-uploading: check the buzzer wiring — one leg to digital pin 8 and the other to GND (see card R5 and the wiring card).

Upload failed: look for red error text in the IDE. If the Serial Monitor is open, close it and upload again.

If still stuck, call the teacher.

V1

Project #2 — Reaction-Time Game: Milestone 6 of 6 • Final milestone

🏆 Write your fastest time on the poster

> *In this milestone you look back over all the rounds you played, find your fastest reaction time, and write it on the Project 2 poster — so the whole group can see how fast you are.*

📋 What to do

- [ ] Open the Serial Monitor and look at the reaction times from all the rounds you played.
- [ ] Find the **smallest number** — that is your fastest reaction time. (Each number is in milliseconds, and smaller means faster.)
- [ ] Go to the Project 2 poster taped at your workstation and find the row with your nickname.
- [ ] In the "fastest time" column, write the number you found, next to your nickname.

- [ ] Call the teacher to celebrate your time together.

👀 WHAT YOU SHOULD SEE

Your nickname is on the poster with your fastest reaction time next to it. The poster is now the group's leaderboard — everyone can see their own time and their friends' times.

You can always play more rounds and try to improve your time — then erase it and write a new, better number.

✅ DONE WHEN

- Your fastest reaction time is written on the poster next to your nickname
- The teacher has celebrated with you

🪄 STUCK?

Not sure which number is the fastest? Remember: the smallest one wins. If the numbers have already scrolled off the Serial Monitor, just play another round or two to get a fresh time. You can always call the teacher for help.

🎉 You just completed Project 2!

You built a real reaction-time game: an LED, a button and a buzzer, with code that measures how fast you are using `millis()` and prints the time to the Serial Monitor. Your time is now up on the poster for you and the whole group to see. Ask the teacher for a photo of your finished build (if you want one) — and start thinking about what you want to build next.

V2

Project #2 — Reaction-Time Game: Milestone 1 of 5

🚀 Start-up — set up the game and play one round

> *A compressed start-up: set up the folder, wire the LED and button, upload the starter sketch, and play one round to confirm the game works. Then take a peek at the two choices waiting for you next.*

📋 What to do

- [ ] **Folder:** open your Project 2 folder and launch Claude Code on it. In the File Explorer sidebar click **Google Drive**, go to **My Drive** → **Arduino_Projects**, open your nickname folder, and create a new folder inside it called **Project_2_Reaction_Time_Game**.

- [ ] **LED:** plant an LED in the breadboard with its two legs in different rows. Connect the long leg (+) to **digital pin 9** through a **220 Ω resistor** (red · red · brown), and the short leg (−) to **GND**.

- [ ] **Button:** plant a push-button **across the centre gap**. One side to **5 V**, the other side to **digital pin 2**, and from that same side also through a **10 kΩ pull-down resistor** (brown · black · orange) to **GND**. (Full details on reference card R1.)

- [ ] **Starter sketch:** open `ino_files/01_wait_flash_measure/01_wait_flash_measure.ino` in the Arduino IDE and click Upload.

- [ ] **One round:** open the **Serial Monitor** (the magnifying-glass icon). Wait for the LED to turn on, press the button as fast as you can, and read your reaction time in milliseconds.

☐ **Look ahead:** read the two choices waiting for you (below).

> 👀 **WHAT YOU SHOULD SEE**
>
> After Upload you get a green "Done uploading" message. The Serial Monitor shows the game's instructions, and after your first press it shows your reaction time in milliseconds.
>
> The game works! In the next milestones you make it
> **YOURS**
> — you choose how it gives feedback and how hard it is, and you change the code yourself with Claude Code's help.

> 🔀 **THE TWO CHOICES WAITING FOR YOU**
>
> - **MILESTONE 2 — HOW THE GAME GIVES FEEDBACK:**
>
> three LEDs (fast / medium / slow), a buzzer pattern, or a rich Serial Monitor readout. Pick one.
> - **MILESTONE 3 — DIFFICULTY:**
>
> easy (the LED stays on for 2 seconds) or hard (the LED stays on for only 0.5 seconds). Here you
> **CHANGE THE CODE YOURSELF**
>
> with Claude Code.
>
> No need to decide now — just read, so you know what is coming. Details for each option are on the reference cards.

> ⚠️ **HEADS UP — YOU CHANGE THE CODE YOURSELF**
>
> In this tier you
> **MODIFY THE CODE**
> (Channel A, Level 2): describe what you want, ask Claude Code, read the answer, edit, and re-upload. It is not copying — you understand what you changed. You will practise this at Milestone 3.

✅ DONE WHEN

- The game is wired (LED on pin 9, button on pin 2 with its pull-down) and running from the starter sketch
- You played one round and saw a reaction time on the Serial Monitor
- You have read the two choices coming up at Milestones 2 and 3

🪄 STUCK?

This milestone packs several small steps into one. If anything is unclear, slow down and do it step by step.

If the Serial Monitor is blank — check it is open and the baud rate is 9600.

If the button does not respond — the pull-down resistor is almost certainly on the wrong side. Check it against reference card R1.

Call the teacher — you can have the detailed Tier 1 cards for any part.

Project #2 — Reaction-Time Game: Milestone 2 of 5

V2

🏁 Pick your feedback mode

> *This is the first design choice of the project. You have already seen the game work — now you choose **how the game tells you your result**: with LEDs, with the buzzer, or on the Serial Monitor. Your choice, no "correct" answer.*

📋 Pick ONE of these three feedback modes

MODE A — THREE LEDS (FAST / MEDIUM / SLOW)

How it works:

an LED lights up to match your speed — a fast reaction lights the **green** LED (pin 9), a medium one the **yellow** LED (pin 10), and a slow one the **red** LED (pin 11).

What you wire:

two extra LEDs, each with its own 220 Ω resistor.

So you have one more wiring card after this one.

Starter sketch:

`ino_files/T2_three_led_feedback_starter/T2_three_led_feedback_starter.ino`

MODE B — BUZZER PATTERN

How it works:

the buzzer plays a different sound for each category — for example one high beep for fast, two beeps for medium, and a low buzz for slow.

What you wire:

nothing new. Use only the buzzer you already wired.

Starter sketch:

`ino_files/T2_buzzer_pattern_starter/T2_buzzer_pattern_starter.ino`

MODE C — RICH SERIAL MONITOR READOUT

How it works:

the Serial Monitor shows a detailed message with a category word — for example `FAST! 210 ms` or `SLOW… 540 ms`.

What you wire:

nothing new. Use only the Serial Monitor.

Starter sketch:

`ino_files/T2_serial_readout_starter/T2_serial_readout_starter.ino`

📋 What to do

- [ ] Read all three modes and think about which one appeals to you most.
- [ ] Pick A, B, or C. Write your choice here: _____
- [ ] **If you chose Mode A** (three LEDs) — your next card is **Milestone 2b**, where you wire two more LEDs.

- [ ] **If you chose Mode B or Mode C** — skip Milestone 2b and go straight to **Milestone 3**.

> 👀 **WHAT YOU SHOULD SEE**
>
> You have clearly marked which feedback mode you chose — A, B, or C — and written it on the card. You know which card comes next.
>
> You do not change any code or upload anything yet — that comes in the next milestones. Right now you are just deciding.

✅ **DONE WHEN**

- You have written your choice (A, B, or C) on this card
- You know where you go next: Mode A → Milestone 2b, Mode B or C → Milestone 3

> 🪄 **STUCK?**
>
> Not sure what to pick? There is no "wrong" choice — they all work. If you like lights, pick Mode A. If you like sounds, pick Mode B. If you like seeing words and numbers, pick Mode C. If you are still torn, call the teacher and choose together.

Project #2 — Reaction-Time Game: Milestone 2b (Mode A) — extra wiring V2

🔌 Wire the three LEDs

> *Only do this card if you picked **Mode A — three LEDs**. You add two more LEDs: medium on pin 10 and slow on pin 11 — same wiring pattern as the LED you already have on pin 9.*

> ⚠️ **SKIP THIS CARD IF YOU CHOSE MODE B OR MODE C.**
>
> Mode B (buzzer) and Mode C (Serial Monitor) need no extra LEDs — your wiring is already done. Go straight to Milestone 3.

📋 What to do

- [ ] Pick two more **LEDs** from your parts tray — in **two colours different** from the first LED (yellow for medium and red for slow work well).
- [ ] Pick two more **220 Ω resistors** (bands: **red · red · brown**) — one per LED.
- [ ] **The medium LED:** plant it on the breadboard, one strip over from the first LED. Each leg in a different row.
- [ ] Connect the medium LED's long leg (+) to **digital pin 10** through a 220 Ω resistor, and the short leg (−) to **GND**.
- [ ] **The slow LED:** plant it on the breadboard, one strip over from the medium LED. Each leg in a different row.
- [ ] Connect the slow LED's long leg (+) to **digital pin 11** through a 220 Ω resistor, and the short leg (−) to **GND**.

- [ ] Check that the LED and button from the earlier milestones are still wired — do not touch them.

Wiring diagram

```
Arduino pin 9  ──[220 Ω]──┬── LED fast (green) long leg
                          ▼
                  LED fast short leg ── Arduino GND

Arduino pin 10 ──[220 Ω]──┬── LED medium (yellow) long leg
                          ▼
                  LED medium short leg ── Arduino GND

Arduino pin 11 ──[220 Ω]──┬── LED slow (red) long leg
                          ▼
                  LED slow short leg ── Arduino GND
```

Project 2 · Circuit 3 — Three LEDs + Button + Buzzer

Tier 2 three-LED feedback · LEDs=D9/D10/D11, Button=D2, Buzzer=D8

- **D9** ──[220 Ω]── FAST LED (green) ── short leg ── **GND**
- **D10** ──[220 Ω]── MEDIUM LED (yellow) ── short leg ── **GND**
- **D11** ──[220 Ω]── SLOW LED (red) ── short leg ── **GND**

- **5 V** ── Button pin A · pin B ── **D2** · pin B ──[10 kΩ]── **GND**

- **D8** ── Buzzer leg 1 · leg 2 ── **GND**

Schematic (preliminary). A photo-style breadboard image will replace this diagram.

Pin 9 → 220 Ω → fast LED (green) · **Pin 10 → 220 Ω → medium LED (yellow)** · **Pin 11 → 220 Ω → slow LED (red)**. All short legs → Arduino **GND**. Same pattern, three pins.

WHAT YOU SHOULD SEE

Three LEDs in three different colours, side by side on the breadboard, each with its own 220 Ω resistor going to a different pin (9, 10, 11). The button is still in place.

NO LED LIGHTS UP YET

— you have not changed the sketch. The code that lights the right LED for your reaction speed comes at Milestone 3.

✅ DONE WHEN

- The medium LED is wired: long leg through a 220 Ω resistor to pin 10, short leg to GND
- The slow LED is wired: long leg through a 220 Ω resistor to pin 11, short leg to GND
- The first LED is still on pin 9, and the button is still in place
- The teacher has confirmed the wiring is correct

🪄 STUCK?

Same wiring pattern as the first LED — just one strip over each time, to pin 10 then pin 11.

The two legs of one LED must be in **different rows** — if they share a row, you create a short.

If the button or first LED stopped working while you were wiring, a jumper wire probably came loose — re-seat it.

Call the teacher if you need help.

V2

💻 Pick difficulty and modify the code

> *This is the second choice point, and the big step of the project: for the first time you change the code yourself, with Claude Code's help. You pick how long the LED stays on — and that sets how hard the game is.*

🎯 Step 1 — Pick a difficulty

Pick **one** of the two. Both are perfectly fine — it is your choice:

- ☐ **Easy:** the LED stays on for **2 seconds**. Slower reactions still count. (This is what the code does now.)
- ☐ **Hard:** the LED stays on for only **0.5 seconds**. Only a fast reaction is quick enough.

In our game we move from "easy" to "hard" — that is, we change the time from 2 seconds to 0.5 seconds. It is one line in the code.

📋 Step 2 — Fill in the (a)(b)(c) on this card before talking to Claude Code

Fill it in first, in pencil. That way you know what you are asking for before you open Claude Code.

```
(a) What I want to happen:
_____________________________________________
_____________________________________________

(b) What is currently happening:
_____________________________________________
_____________________________________________

(c) What I have tried (or looked at):
_____________________________________________
_____________________________________________
```

💻 Step 3 — Send the prompt to Claude Code

In the Claude Code window, paste this prompt, and paste your current sketch code where it says [paste]:

```
Here is my current delay. How do I make the LED stay on for only
0.5 seconds instead of 2 seconds? Here is my code:
[paste]
```

📋 Step 4 — Read the answer, change one line, upload, test

- [ ] Read Claude Code's response carefully. Find the **one line** that sets how long the LED stays on (the delay number).
- [ ] Open your sketch in the Arduino IDE and change **only that number** — leave the rest alone.

- [ ] Click Upload.
- [ ] Play a round: does the LED stay on for a shorter time now? Does it feel harder?
- [ ] If nothing changed, go back to Step 2 and describe what happened more specifically.

📋 Comprehension check

Answer this in one sentence (out loud, or write it below):

```
After Claude Code helped me, the line I changed in my code was:
_____________________________________________________
_____________________________________________________
```

👀 WHAT YOU SHOULD SEE

The LED lights up for the time you chose — 0.5 seconds (hard) or 2 seconds (easy). The game feels different: on hard you have less time to press.

You can point at one line of code and say "this is the line that sets how long the LED stays on."

✅ DONE WHEN

- [ ] You picked a difficulty — easy or hard
- [ ] You filled in (a), (b), and (c) on this card before talking to Claude Code
- [ ] You changed how long the LED stays on, and the change works when you play
- [ ] You can say in one sentence which line you changed

> 🪄 **STUCK?**
>
> If Claude Code's answer did not work: don't panic. Go back to (a)(b)(c) and describe what happened more specifically. "It didn't change" is less useful than "the LED still stays on for a long time like before, not shorter." If you are still stuck, call the teacher.

Project #2 — Reaction-Time Game: Milestone 4 of 5

V2

💻 Upload, test, and tune your variant

> *Last milestone you changed your sketch with Claude Code. Now you upload it, play five rounds, and check the game runs exactly the way you wanted — your chosen feedback mode and your chosen difficulty.*

⬆️ Step 1 — Upload your sketch and play 5 rounds

- [ ] Open your updated sketch in the Arduino IDE and click Upload. Wait for the green "Done uploading" message.
- [ ] Open the **Serial Monitor** (the magnifying-glass icon) so you can see your reaction times.
- [ ] Play **5 rounds**: wait for the "go" light, press the button as fast as you can, and read your reaction time.
- [ ] Check that your chosen feedback mode works — the three LEDs (Mode A), the buzzer pattern (Mode B), or the Serial Monitor message (Mode C).
- [ ] Check that your chosen difficulty is right — the LED stays on for 2 seconds (Easy) or only 0.5 seconds (Hard).

📋 Step 2 — Tune the thresholds (only for Mode A or Mode B)

If you picked **Mode A (three LEDs)** or **Mode B (buzzer pattern)**, the game sorts every reaction time into **fast / medium / slow**. Maybe the thresholds do not feel fair to you — for example, everything comes out "slow". You decide what feels fair for you.

```
"Fast" for me = under _______ milliseconds
"Medium" for me = between _______ and _______ milliseconds
"Slow" for me = over _______ milliseconds
```

If you want help changing the thresholds in the code, this is one more small Claude Code ask (Level 2). Fill in the (a)(b)(c) first, then ask:

```
I am working on my Project 2 reaction-time game.

(a) What I want: a time under 250 ms should count as "fast".
(b) What is happening now: almost everything comes out "slow".
(c) What I have tried: I played 5 rounds and wrote down my
times.

Here is my current code:
[paste the whole sketch file here]

Can you help me change the fast/medium/slow thresholds?
```

👀 WHAT YOU SHOULD SEE

The game runs the way you chose: your feedback shows after every press, and the LED stays on for the difficulty you picked. If you tuned the thresholds, a fast press comes out "fast" and a slow press comes out "slow".

You can show your variant to the teacher and say "this is the game I built."

✅ DONE WHEN

- The updated sketch is uploaded and you have played 5 rounds
- Your chosen feedback mode works (three LEDs / buzzer / Serial Monitor)
- Your chosen difficulty is right (LED stays on for 2 seconds or 0.5 seconds)
- (Mode A/B) the fast/medium/slow thresholds feel fair to you

🪄 STUCK?

If the buzzer or LEDs do not react, the updated sketch may not have uploaded — upload again and wait for "Done uploading".

If the thresholds still do not feel fair after a few tries, go back to (a)(b)(c) and tell Claude Code exactly what is happening — for example "a 300 ms press is counted as slow and I want it to be medium."

If it still does not work, call the teacher.

V2

Project #2 — Reaction-Time Game: Milestone 5 of 5 • Final milestone

🏆 Signature game — name it and share

> *In this milestone your variant already works exactly how you wanted — now you give it a name, write your fastest time on the poster, and play a round head-to-head against a peer or the teacher.*

🏷️ Step 1 — Name your game

Your variant — the feedback mode and difficulty you chose — is your **signature game**. Give it a name that fits it (for example "Lightning Round", "Owl Mode").

```
My reaction-time game is called:
_____________________________________________

My fastest time (in milliseconds):
_____________________________________________
```

📋 Step 2 — Record it on the poster

- [ ] Find the **Project 2 poster** taped at your workstation.
- [ ] On the **Tier 2 line** of the poster, write your **game name** and your **fastest time**.
- [ ] If you are not sure of your fastest time, play a few more rounds and check the numbers on the Serial Monitor.

🎮 Step 3 — Play a round against someone

- [ ] Invite a peer from the group, or the teacher, to play one round of your signature game against you.
- [ ] Explain the rules in a sentence or two: when to press, and what the feedback tells you (the LEDs / the buzzer / the Serial Monitor).
- [ ] Play one fun round — it does not matter who wins, what matters is that someone else saw your game working.
- [ ] If you want, ask the teacher to take a photo of your game (only if you agree). You can keep it for your portfolio or show family.

👀 WHAT YOU SHOULD SEE

Your version of the reaction-time game running on your breadboard with the name you chose, your fastest time written on the Tier 2 line of the poster, and someone else having played a round against you.

The teacher has seen the game and celebrated with you, and you have a photo (if you wanted one).

✅ DONE WHEN

- You gave your game a name
- You wrote the name and your fastest time on the Tier 2 line of the poster
- You played a round against a peer or the teacher — someone else saw your game

🪄 **STUCK?**

If it is hard to decide on a name, think about how your game feels: fast? tough? funny? Even a simple name like "[Your name]'s game" counts completely. If no peer is free to play against you right now, the teacher is always happy to be your opponent. Call the teacher if you would like help.

🎉 You just completed Project 2!

You picked a feedback mode, picked a difficulty, modified code with Claude Code yourself, and built a reaction-time game that is your own — with a name, a best time, and someone who saw it working.

Project #2 — Reaction-Time Game: Project Planner

V3

🏁 Design your own reaction-time game

> *You design your own version of the reaction-time game. Pick a variant — a two-player head-to-head game, or a multi-round scored game across five rounds — and build it. Fill in the planner, build, show.*

📋 Phase 1 — Plan (PLAN)

WHICH VARIANT DO YOU CHOOSE? CIRCLE ONE:

*(1) **Two players, head-to-head:** add a second button (on **digital pin 3**) with its own pull-down resistor. The LED turns on, the two players race to press first, and the game shows who won.*
*(2) **Multi-round scored game across 5 rounds:** one player plays five rounds, the game keeps score (for example a total reaction time, or points by speed), and shows the final result at the end.*

DESCRIBE THE GAME FLOW IN YOUR OWN WORDS. WHAT HAPPENS FROM START TO FINISH?

This is the most important field. Be as specific as you can. Example (two players): "The LED waits 2–5 seconds and turns on. The first player to press wins the round. If someone presses before the LED is on, they lose the round. The buzzer plays a win beep, and the Serial Monitor prints who won."

HOW DOES THE GAME SHOW THE RESULT? (FEEDBACK)

Choose: an LED lights for the winner, a buzzer beep, a message on the Serial Monitor, or a mix. You can use the go LED, the buzzer, and anything already on the breadboard.

WHAT HARDWARE DO YOU NEED? (BUTTONS, LEDS, RESISTORS, BUZZER, JUMPER WIRES — HOW MANY OF EACH)

The two-player variant needs an extra button and an extra 10 kΩ pull-down resistor. The scored variant usually needs no new hardware at all. If you need something unusual, ask the teacher before you start.

Phase 2 — Build (BUILD)

SKETCH YOUR WIRING ON PAPER, THEN WIRE IT ON THE BREADBOARD.

*Your wiring is a variation on the game you already built. For the two-player variant: add a second button **across the centre gap** — one side to **5 V**, the other side to **digital pin 3** and through a **10 kΩ pull-down resistor** (bands: **brown · black · orange**) to **GND**. Exactly like the first button, just on pin 3.*

Project 2 · Circuit 4 — Two-Button Variant (Tier 3)

Head-to-head game · Button 1=D2, Button 2=D3, LED=D9, Buzzer=D8

- **D9** —[220 Ω]— LED (+) ; short leg — **GND** · **D8** — Buzzer — **GND**
- **Player 1:** 5 V — Button 1 pin A ; pin B — **D2** ; pin B —[10 kΩ]— **GND**
- **Player 2:** 5 V — Button 2 pin A ; pin B — **D3** ; pin B —[10 kΩ]— **GND**

Each player has their own button; the sketch shows who pressed first.

Schematic (preliminary). A photo-style breadboard image will replace this diagram.

Two-player variant: player 2's button on **pin 3** with its own pull-down resistor (10 kΩ) to GND, alongside the existing button on pin 2. The LED and buzzer stay on pins 9 and 8.

TAKE A PHOTO OF YOUR FINISHED WIRING. NOTE HERE: *PHOTO TAKEN / NOT YET / I DON'T WANT ONE*

💻 Phase 3 — Code (CODE)

You use Claude Code in free-dialogue mode here. You describe what you are trying to build, Claude Code helps you write the sketch, and you iterate together until it works. The same (a)(b)(c) discipline still applies — describe the problem before asking for a solution. There is no pre-written sketch here — you build it from your description.

FIRST PROMPT TO CLAUDE CODE (DRAFT IT HERE BEFORE YOU SEND IT)

Example (two players): "I am building a two-player reaction-time game. An LED on pin 9, a buzzer on pin 8, player 1's button on pin 2 and player 2's button on pin 3, each with a 10 kΩ pull-down. After a random wait I want the LED to turn on, and the first player to press wins the round — and I want the Serial Monitor to print who won. Can you help me write the Arduino sketch?"

📋 Phase 4 — Test (TEST)

DOES IT DO WHAT YOU PLANNED?

If no: what is different? Write it down, go back to Claude Code with a new (a)(b)(c), iterate. The plan can change as you go — that is completely fine.

DOES IT SOMETIMES WORK AND SOMETIMES NOT? WHAT DID YOU NOTICE?

Intermittent bugs are usually either wiring (a loose jumper, or a missing pull-down on the second button) or timing (a delay that is too short or too long, or someone pressing too early).

📋 Phase 5 — Show (SHOW)

- [ ] When you are happy with it (or when you have decided it is as good as it is going to be today), call the teacher.

- [ ] Show your game — ideally play a real round against a friend or the teacher. Explain in one or two sentences what it does and why you built it this way.

- [ ] If you want, take a photo of the finished game and save it to your Project 2 folder on Google Drive.

> 🪄 **STUCK AT ANY PHASE?**
> This planner is open-ended and that is its point. Being stuck is part of designing your own game. Use Claude Code to think through the problem, and call the teacher when you feel like you have been going in circles on the same thing. A teacher conversation here is a design conversation, not a troubleshooting one — tell them what you are trying to build and what is not working.

🎉 You just completed Project 2!

You designed, built, and programmed your own variant of the reaction-time game from scratch.